A direct route to internal consistency of PTS-rk

October 22, 2026

Abstract

We propose a direct, resource-relativized consistency proof in the original PTS-rk, without interpreting $H_{<\omega}$ or constructing an Elementary Arithmetic derivation. The starting points are the independent native certificate checker, external graded normalization and canonicity, and the new internal PHOAS fold interface. The principal missing construction is one fixed-rank internal soundness term for arbitrary represented certificates with independently certified resources. A continuation-encoded resource domain then gives a short negative consistency proof. This is a roadmap, not a claim that internal soundness or consistency has already been proved.

1 Recommendation and current starting point

Prioritize **proof-local normalization and relational contradiction separation**, not full finite models of every rank fragment. This uses the finite resources needed by a supplied proof and avoids stronger semantic closure obligations that are unnecessary for a negative consistency result.

There is an important distinction in the current combinatory note[1]. Its checked result is contextual soundness in *graded finite syntactic carriers*: application can enlarge the grade, and valuations are typed syntactic substitutions. It is an Agda metatheorem, not yet an internal PTS soundness term or a fixed finite first-order model. The latter would require total operations on fixed domains and validity under every semantic valuation. Neither can be inferred from the former.

The useful existing interfaces are:

- `NativeCertificates`: independent six-rule certificates, explicit annotated beta instructions, `accepted-typing`, `certify`, and `certificate-adequacy`.
- `NativeRankCoverage`, `NativeNormalization`: computed sufficient bounds, total typed normalization, conversion adequacy and budget independence.
- `FragmentModelSoundness`, `FragmentPackages`: soundness for independently accepted contextual inputs and joint finite bounds for represented data.
- `NativeCanonicity`: `normal-arbitrary-type-empty`, `normal-Void-empty`, and `accepted-Void-rejected`, all external Agda results.
- `ChurchFinite.v`: originally typed selectors, unary tables and iterations, with beta equations. Generic binary correctness and correspondence with the Agda generators remain separate obligations.
- `Algebra`, `GenericFold`, `UnifiedAlgebra`: actual original PTS folds, ordinary algebra arguments, and tag-free dependent identity interpretation. Their signatures still vary externally; they do not internalize acceptance.

The Agda results are specifications and regression oracles for the new construction. Porting a theorem to Rocq is useful, but still does not make its conclusion an inhabitant of an original PTS type.

[1] `../CombinatoryLogic-rk/CombinatoryPTS-rk.tex`

2 Freeze a direct consistency statement

Write $T = \mathsf{PTS}_{rk}$ and $\bot = \Pi X : U_0.X : U_1$. All final terms use the original annotated syntax, typing and beta conversion: no primitives, cumulativity or eta rule.

Choose a homogeneous structural code type D for complete finite certificates. A code includes its context data, annotations, rule ranks, conversion instructions and any checker-trace witnesses. Thus quantifying over the whole code also quantifies over every trace; no trusted trace is silently supplied by the metatheory. A finite constructor alphabet can represent arbitrary binary labels as trees without turning those labels into actual universe literals during interpretation.

Define an independent predicate $\mathsf{Acc}_\bot(q)$ meaning that the raw certificate checks and its conclusion is a closed term of the displayed syntactic type $\bot$. It must contain local syntax/graph checks, not native `Typing.typ`, successful normalization, or a semantic soundness assumption. The proposed source is the existing native certificate grammar. It is not the draft's distinct T^e calculus with typed judgmental equality. Develop its general judgment checker first and specialize its root to $\bot$ afterward. Positive acceptance tests and adequacy must cover ordinary inhabited judgments, not only the externally empty contradiction case.

Choose another fixed structural carrier B for resource descriptions and an independently defined predicate $\mathsf{Res}(q, b)$. It certifies well-generated data, size/rank envelopes and sufficient finite capacity. It must not assert rejection of q, existence of a normalizing history, or validity of a model of the source theory. These are conclusions to prove. Here well-generated means structural code validity, not acceptance or well-formedness of the encoded typing context; rejected standard inputs must also admit resources.

All carriers, graph predicates and proof types must be well formed at **one fixed finite collection of PTS ranks**, independent of the represented source ranks, certificate lengths and budgets. This is a construction obligation, not a consequence of representing a number by a term.

2.1 The central obligation and a short final assembly

The central target, ($\mathrm{BS}_{\mathrm{dir}}$), is an actual closed original PTS term

$$\mathsf{bs} : \Pi q : D.\Pi b : B.\mathsf{Res}(q, b) \to \mathsf{Acc}_\bot(q) \to \bot.$$

Define the admitted domain by a fixed-answer-universe resource package:

$$\mathsf{Adm}(q) := \Pi Z : U_1.(\Pi b : B.\mathsf{Res}(q, b) \to Z) \to Z,$$
$$\mathsf{Cons}_{\mathrm{dir}} := \Pi q : D.\mathsf{Adm}(q) \to \mathsf{Acc}_\bot(q) \to \bot.$$

Since $\bot : U_1$, the proposed final proof is simply

$$\lambda q\, a\, h.\ a \bot (\lambda b\, r.\mathsf{bs}\, q\, b\, r\, h).$$

This is a proof plan, not an assumption of bs. Every displayed type still needs its original formation derivation. The package permits elimination into $\bot$ without extracting a positive resource witness from double negation. Neither a uniform function choosing a budget nor a normal-form output inside the admitted domain is needed.

Explicit b and $r : \mathsf{Res}(q, b)$ introduce admission by $\lambda Z\, k.k\, b\, r$. There is also a concrete formation target: if $D : U_d$, $B : U_e$, $\mathsf{Res}(q, b) : U_r$ and $\mathsf{Acc}_\perp(q) : U_s$, the original product-rank rule gives

$$\mathsf{Adm}(q) : U_{\max(e+2,r+2)},$$
$$\mathsf{Cons}_{\mathrm{dir}} : U_{\max(d+1,e+3,r+3,s+1)}.$$

These are schematic rank calculations to verify on the actual annotated builders, not newly checked Rocq theorems.

To make this a meaningful consistency interpretation, prove separately:

1. Every standard finite **raw** certificate, accepted or not, has an encoding in the same D and an Adm proof. Construct explicit finite resources; do not use external consistency to establish admission.
2. On canonical codes, acceptance agrees with the independently specified checker, including all conversion and trace data. Its internal laws must also hold for arbitrary guarded inputs, not only canonical examples. In particular, construct an internal acceptance proof for each accepted standard certificate, not just a Boolean computation with value true.
3. The source certificate system covers every original typing derivation. Any conversion to another certificate format preserves the relevant predicates and resource guards in the direction being used.

This specifies a resource-relativized structural consistency sentence. It is not automatically the old $J(\mathcal{NAT}(C))$, nor unrestricted arithmetical consistency. If separate proof and trace variables are retained, prove the joint-resource lemma from their individual guards. Packaging the entire input is a deliberate choice of sentence, not a free equivalence.

3 What to retain from the two papers

Pakhomov's Lemma 4.1 uses only the axioms and types occurring in a supplied proof, with enough witnessed resources to interpret its subproofs. Lemma 3.11 and Corollary 5.3 provide transfer results for predicate-only Π_1 sentences; they are not PTS theorems. Section 5's bounded relational pair interpretation addresses missing resource margins without requiring total successor. We retain this proof-local and relational discipline, but replace the transfer step by the original PTS construction ($\mathrm{BS}_{\mathrm{dir}}$), not by another citation. Primary source: Pakhomov, v2[2].

From the supplied *Zermelo revision*, retain these specific devices:

- `lem:rank-compression`: rename all indices in the complete certificate, preserving the product equations actually used and fixing $0, 1$.
- `eq:good`: constructor-generated, bounded-rank induction certificates, defined independently of observation equality or soundness.
- `eq:delta`, `lem:internal-finite`: guarded finite elimination. Open a target domain guard before eliminating its arguments, so the motive remains at the permitted rank.
- `eq:inv`, `lem:inv`: continuation packaging of positive domain evidence together with stable relational cases. Do not extract positive evidence from a negative conjunction.
- `prop:adequate-height`, `thm:bs-proved`: reuse a supplied resource prefix, account for proof transport as well as reduction, and absorb fixed overhead without demanding a total variable-height tower function.

[2] `https://arxiv.org/pdf/1907.00877v2`

- `sec:internal-sn-scope`: a negative consistency target need not return witnesses inside the interpreted domain.

These are design templates from an unformalized manuscript. Their rank, scope and cost claims must be proved for the actual new encoding. We do not import the draft's conclusion as an axiom or change the original PTS rules.

4 Roadmap and acceptance gates

4.1 Independent certificates and the exact target

Port the native six-rule certificate format to Rocq, including raw equality trees with explicit redex locations. Prove decoding, scope correctness, acceptance-to-original-typing and source completeness. Preserve all lambda annotations. Conversion verification uses supplied instructions, not a normalization oracle disguised as a cheap checker.

Fix the complete structural serialization and a size measure L charging all contexts, annotations, ranks and witness trees. State explicitly whether shared subdata are charged once or on expansion. Specify D, B, acceptance, resource validity and the final sentence before building soundness.

Gate: all six rule cases have independent acceptance clauses; malformed contexts, wrong substitutions, mismatched conversion endpoints and forged rank data are rejected. No acceptance premise is the desired interpretation. If the draft's exact T^e sentence is required later, add checked certificate translations and guard comparison rather than appealing to informal equivalence of typing systems.

4.2 Elementary compression of source ranks

Collect every sort label in the complete certificate, including contexts, annotations and equality paths, together with all rule ranks and used computed product-result ranks. Adjoin $0, 1$. Let σ enumerate the distinct labels in increasing order. For every used product equation,

$$c = \max(a + 1, b) \quad \Longrightarrow \quad \sigma(c) = \max(\sigma(a) + 1, \sigma(b)).$$

If $c = a + 1$, no integer can lie between the two labels, so their new indices are consecutive. If $c = b > a$, strict order suffices. This is not a claim that arbitrary increasing renamings preserve successor everywhere. Prove that renaming commutes with weakening, substitution and annotated beta steps; lift it through the entire accepted certificate. It fixes $\bot$.

The number of new labels is linear in L. Compressed terms have unchanged tree shapes, and the existing sufficient redex-bound expression $1 + \mathsf{maxRankLiteral}(t) + \mathsf{nodes}(t)$ becomes $O(L)$ for the displayed source terms. Thus the existing elementary bounds suggest a tower envelope of polynomial height in complete input size, without paying for the numeric magnitude of binary labels. Bounds for constructed proofs, substitutions and histories must still be established separately.

Gate: prove both the external original-typing theorem and an internal relational compression theorem on guarded codes. Source-rank compression does not compress an externally indexed family of target types to one fixed-rank internal interface; that is a different obligation.

4.3 Internal guards and a genuinely internal finite pilot

For a fixed homogeneous code carrier C, use a predicate of the form

$$\mathsf{Good}_r(x) := \Pi P : C \to U_r.\ \mathsf{Closed}_{\text{constructors}}(P) \to P(x).$$

Expand the finite constructor-law telescope into ordinary arguments. Prove constructor certificates and the exact rank of every motive used. Never assert that all raw Church-code inhabitants satisfy this predicate. Use exact-rank padding where necessary, not cumulativity.

First test the approach at one fixed finite bound. For selector carrier E_m and canonical selectors $c_1, \ldots, c_m$, a guard

$$\mathsf{Can}_{m,r}(x) := \Pi P : E_m \to U_r.\ P(c_1) \to \cdots \to P(c_m) \to P(x)$$

supplies the precise finite-case elimination missing from beta equations on selectors alone. Internally prove table laws on **all guarded inputs**, including generic binary tables if the implementation uses them. Open the target continuation or guard before eliminating inputs. Keep positive well-formedness evidence available explicitly.

Gate: obtain an originally typed bounded rejection term for the independent checker, not just an externally verified truth table. Every case must be justified by local checking/inversion, without invoking the global external `Void-empty` theorem as the source of rejection. This is a finite pilot, not the final all-budget theorem; its varying E_m types must not be passed off as one internally quantified carrier.

4.4 One structural resource interface

Represent budgets, finite sequences, table entries and schedules as data in fixed carriers. A varying finite dictionary or table must be a value of a fixed type, not an externally generated telescope with an unbounded number of fields. Define relations for lookup, prefix, size comparison and resource extension, with explicit witnesses when an operation is partial.

An envelope must account for the input, compressed rank bound, annotations, substitution images, reduction states, conversion data, proof transport and the entire written history. Closure facts concern these independently specified operations. Neither arbitrary semantic table correctness nor existence of a successful normalization history belongs in the envelope.

Prove admission of every standard raw input by constructing a sufficiently large finite resource diagram. Internally derive exactly the prefix, subcertificate and joining laws used in soundness. Do not silently demand closure of the admitted domain under every successor, proof concatenation or tower construction. If the representation needs a fixed amount of spare capacity, supply it in the resource contract; paired relational resources are an optional repair requiring their own correspondence proof.

Gate: all formulas and induction motives have a fixed, explicit rank budget. All resource operations required by the subsequent proof work on arbitrary guarded inputs. Finite-table generation at each external bound does not pass this gate.

4.5 Internal proof-local soundness

Replay the graded syntactic argument inside the original PTS. Carry an invariant over represented proof nodes containing scoped context, subject and type data, the checked rank

information, substitution environments, and the finite resource bounds needed by the next operation. Its definition must be independent of the final contradiction theorem.

Construct each local closure proof: sort, variable, product, abstraction, application and conversion, plus binder lifting and annotated substitution. For conversion, transport along every supplied equality instruction, not only along traces manufactured by the quoter. Combine these using the proved guarded induction/fold principle. The result must hold for arbitrary accepted code arguments. Raw intermediate terms in an equality certificate need not be typed or normalizing. Normalize the appropriately typed subject/type, not every term appearing in a beta zigzag; equality transport and separation must handle the supplied instructions without that extra premise.

Then internally derive a bounded reduction history to a normal root and preserve the represented typing certificate along it. A verifier for a supplied history is insufficient: its existence or a negative continuation eliminating the need to expose it must follow from acceptance and the independent resource envelope.

The current raw-state bounds do not bound dependent proof transport. Audit the draft's proposed recurrence, for one fixed d, or prove a replacement:

$$s_0 \le M, \qquad K \le M, \qquad s_{i+1} \le (s_i + M + 2)^d \quad (i < K).$$

This gives a fixed elementary envelope in the *supplied* M; charging a fresh exponential level for every beta step would not. The constant d must be independent of the represented source rank and proof length. Uniformity of the actual algorithm in this representation is a new proof, not a generic elementary-function compiler already available in the repository.

Gate: an original PTS derivation of witnessed history soundness, with all local premises and size/rank costs accounted for. If full proof transport violates the resource estimate, use the draft's active graded calculus as a fallback: its product projections, source elaboration, constant separation and terminal lemma then become explicit new obligations. Do not develop that parallel machinery unless this gate needs it.

4.6 Small terminal separation and ($\mathrm{BS}_{\mathrm{dir}}$)

Internalize the normal-form obstruction for $\bot$. A normal closed function of type $\Pi X : U_0.X$ must be a lambda; its body would be a normal term of type X in the context $X : U_0$. Constructor and neutral-head inversion exclude that judgment. Represented conversion cannot identify the required sort, product and variable heads; prove the necessary separation and product-injectivity facts internally, with resource bounds.

`NativeCanonicity` provides the external specification, not an internal axiom. In the fallback active calculus, the weaker grade-zero weak-head terminal obstruction may replace full normality.

Gate: compose guarded acceptance, resource-bounded history construction and terminal separation into the single fixed-type term ($\mathrm{BS}_{\mathrm{dir}}$). No source-rank-indexed family of final terms is sufficient. The consistency proof then uses the short resource-package elimination already displayed. Its output is literal $\bot$, not a code of falsehood without an internal falsehood bridge.

4.7 Adequacy, relational comparison and the incompleteness audit

Audit standard raw-code admission and checker agreement independently of soundness. For a conventional predicate-only presentation, interpret atomic relations by the proved graph

predicates and quantify every trace witness. If returning to the historical sentence, prove certificate translation, domain correspondence, resource joining and the implication needed to transport consistency internally. Agreement on standard numerals alone does not identify two internal statements.

Run the logical viability audit early, alongside steps 1–4. Avoiding arithmetic notation does not avoid Gödel's second incompleteness theorem. If this coding supports the applicable diagonal fixed point and the Hilbert–Bernays–Löb conditions, a consistent theory cannot prove the corresponding consistency statement. Determine which domain/closure or derivability conditions are actually available; do not simply declare one to fail. A relational presentation and separate admission of every standard code do not automatically supply uniform successor or proof-combination closure. Conversely, their absence cannot merely be assumed. Pakhomov's introduction discusses this distinction[3].

Gate: the final claim names its precise interpretation. A proof only for externally bounded codes is reported as a schema; a resource-relativized proof is not relabelled unrestricted self-consistency. If the chosen domain forces the usual obstruction, revise the independent interpretation or record a conditional no-go result, without hiding the target behind an acceptance predicate that already assumes soundness.

5 The self-interpreter's precise role

The common PHOAS fold supplies typed specialization/application once its algebra fields are constructed; compatible operations can then be composed or iterated with separate typing proofs. Use it for representation operations and, if feasible, the proof-node invariant algebra. `GenericFold.run_computes` does not establish those fields' soundness, and its finite signature is still chosen externally. A fixed signature for the structural certificate language is a narrower target than a rank-only algebra for all dependent PTS programs.

An alternative finish would construct

$$\mathsf{Res}(q, b) \to \mathsf{Acc}_{\perp}(q) \to \mathsf{Code}_a(\perp)$$

at one fixed a and apply the existing typed decoder. This is useful only if it simplifies step 5. `FixedInterface.inhabited_iff` shows why it is not a shortcut around soundness: the output code is inhabited exactly when $\perp$ is. External quotation from an already supplied native derivation does not construct this internal map.

The primary route needs no raw HOAS tag, intrinsic all-source syntax injectivity, CPS theorem, literal unquotation of arbitrary universe labels, or native completeness of the 65-shape Hilbert catalogue. Start with the complete annotated certificate grammar and use a binder-free tree only as a serialization of its nodes. A later switch to genuinely combinatory proof certificates needs a native-source completeness bridge.

Likewise, numeral conversion is optional. Structural codes still require size, order, iteration and resource relations, but these need not pass through an arithmetical theory. The literal-alphabet obstruction concerns unbounded strong unquotation, not this fixed-result relational proof.

[3] `https://arxiv.org/pdf/1907.00877v2`

6 Proposed Rocq work packages

The names below are planned modules in `PTSrk.SelfConsistency`, not existing proofs. Register the library only when there are actual definitions and proofs. Each internal package must expose a literal PTS term, its original type and a derivation using `Typing.typ` and `Ranks.RankSorts`.

1. `NativeProofCodes`, `CertificateAdequacy`: independent source grammar, exact target, external decoder/checker correspondence and completeness.
2. `RankCompression`: complete-certificate renaming and its resource bound; then the internal graph/certificate counterpart.
3. `GuardedCodes`, `GuardedFiniteTables`, `ResourceInterface`: originally typed induction/case guards, a real finite pilot, and the fixed-type resource language. Audit motive ranks before advancing.
4. `InternalSubstitution`, `InternalProofInvariant`, `BoundedHistory`: all local rules and conversion, guarded induction and witnessed normalization with bounded proof transport.
5. `InternalNormalBottom`, `DirectBoundedSoundness`, `DirectConsistency`: internal terminal separation, ($\mathrm{BS}_{\mathrm{dir}}$), and resource elimination.
6. `DomainAdequacy`, `RelationalComparison`, `Audit`: standard admission, any needed comparison to the old target, original formation/typing and fresh transitive assumption reports against pristine upstream.

The first practical milestone is **not another global normalizer**: implement the native certificate interface and rank compression, while testing guarded internal finite-table correctness in parallel. These expose the encoding and rank risks before investing in the full bounded-history proof. The decisive milestone remains one internal term for arbitrary represented budgets; it replaces, rather than assumes, the old transfer.

7 Provenance and verification boundary

This roadmap supersedes the *priority order* of the earlier two-route summary[4], not its record of the draft's original target. It uses the supplied Zermelo revision's named lemmas and the current repository interfaces listed above. The Pakhomov comparison is motivational and methodological: no unverified transfer theorem enters the proposed checked dependency graph. Neither EA nor $H_{<\omega}$ is an intermediate proof theory in this plan.

No new Rocq or Agda theorem is claimed here. Documentation builds, source identity checks and the exact review record are recorded in `EVIDENCE.md`.

[4] `README.tex`